

A Practitioner’s Guide to Deep Learning for Individual Heterogeneity

Pranjal Rawat*

Sanjog Misra†

February 2026

Abstract

This paper provides a step-by-step guide to implementing the deep learning framework for individual heterogeneity developed in [Farrell et al. \(2021\)](#) and [Farrell et al. \(2025\)](#). We show how neural networks can directly estimate heterogeneous structural parameters from individual-level data, and how influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. Our running application uses H&M fashion transaction data: we train two-tower contrastive embeddings to represent consumer preferences, estimate a multinomial choice model with heterogeneous price sensitivity, and construct influence-function-corrected confidence intervals for average elasticities and counterfactual pricing experiments. We provide complete, documented code and a companion `Python` package (`deep-inference`) that handles cross-fitting, Lambda estimation, and influence function assembly automatically. The guide is intended for applied researchers and practitioners who want to move beyond assuming homogeneous effects without sacrificing valid inference.

Keywords: deep learning, individual heterogeneity, influence functions, multinomial choice, demand estimation, contrastive learning

JEL Codes: C14, C45, D12, L81

*University of Chicago Booth School of Business. Email: pranjal@chicagobooth.edu

†University of Chicago Booth School of Business. Email: sanjog.misra@chicagobooth.edu

Contents

1	Introduction	4
1.1	The Idea in Brief	4
1.2	Running Example: H&M Fashion Demand	5
1.3	What This Guide Covers	5
2	The Framework	6
2.1	Setup	6
2.2	Target Functionals	6
2.3	Why Naive Inference Fails	7
2.4	The Influence Function Correction	8
2.5	Three Regimes for Lambda	8
2.6	The Multinomial Choice Model	9
2.7	Available Models and Targets	9
3	Estimation in Practice	10
3.1	Data Requirements	10
3.2	The Algorithm	11
3.3	Cross-Fitting Details	11
3.4	Lambda Estimation	12
3.5	Network Architecture	13
3.6	Diagnostics	13
3.7	Common Pitfalls	14
3.8	Quick Start: Binary Logit	14
4	Application: H&M Fashion Demand	15
4.1	Data	15
4.2	Learning Representations	15
4.3	Mapping to the FLM Framework	16
4.4	Results	17
4.4.1	Simulation Validation	17
4.4.2	Main Inference Results	17
4.4.3	Heterogeneity in Price Sensitivity	18
4.5	Counterfactuals	18
5	Concluding Remarks	19
5.1	Other Structural Models	19
5.2	Other Targets	20
5.3	Custom Models via <code>model_from_loss()</code>	20
5.4	Computational Considerations	20

5.5	Three Regimes Revisited	20
A	Implementation Details	23
A.1	Installation	23
A.2	Data Preparation	23
A.3	Inference Call	24
A.4	Running All Parameters	25
A.5	Diagnostics Interpretation Guide	26
A.6	Companion Repository	26

1 Introduction

Individual heterogeneity is ubiquitous in economic data. Consumers differ in their sensitivity to prices, workers respond differently to job training, and patients vary in their treatment effects. Empirical researchers have long recognized this heterogeneity and built it into their models—from random coefficients logit (Berry et al., 1995; Nevo, 2000) to varying-coefficient models more broadly. But estimating heterogeneous parameters at the individual level, and doing valid statistical inference on the resulting objects, has remained a difficult task. Traditional approaches typically assume a parametric distribution for heterogeneity (e.g., normal random coefficients) or restrict attention to a small number of discrete types.

This paper is a practitioner’s guide to a different approach. Farrell et al. (2021) and Farrell et al. (2025) show how deep neural networks can directly estimate individual-level structural parameters $\theta(x)$ from observational data, and how influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. The theoretical foundations are in their *Econometrica* papers. Our goal here is to make the method accessible and practical.

The parallel to Nevo (2000) is intentional. Just as Nevo bridged the gap between the BLP theory papers (Berry et al., 1995) and applied practice, we aim to bridge the gap between the FLM theory and applied implementation. We walk through the entire pipeline—from data preparation to model specification to inference to counterfactuals—using a running example that illustrates every step. We provide complete code, a companion Python package (`deep-inference`), and a simulation study validating the method.

1.1 The Idea in Brief

Consider a structural model with individual-level parameters. A researcher observes outcomes Y_i , treatments or product characteristics T_i , and individual covariates X_i for $i = 1, \dots, n$ observations. The structural model specifies a loss function $\ell(Y_i, T_i, \theta)$, and the individual’s true parameters $\theta^*(X_i)$ minimize the expected loss conditional on their type X_i .

The key insight of FLM is: let a neural network directly output the structural parameters. Train a network $\hat{\theta}(X_i)$ by minimizing the structural loss across observations. Then use influence functions to correct for the regularization bias that neural networks inevitably introduce.

Why is correction necessary? Neural networks use regularization (weight decay, dropout, early stopping) to prevent overfitting. This regularization shrinks estimates toward zero, making naive standard errors—computed from the Hessian of the structural loss—systematically too small. The resulting confidence intervals severely undercover. In our simulations, naive coverage can fall below 20% when the nominal level is 95%.

The influence function (IF) correction accounts for the estimation noise in $\hat{\theta}(X)$. The corrected

estimator for a target functional $\mu^* = E[H(X, \theta^*(X), \tilde{t})]$ is:

$$\hat{\psi}_i = H_i - H_{\theta,i} \cdot \hat{\Lambda}_i^{-1} \cdot \ell_{\theta,i} \quad (1)$$

where H_i is the target evaluated at the estimated parameters, $H_{\theta,i}$ is the Jacobian of the target with respect to θ , $\hat{\Lambda}_i$ is the expected Hessian of the loss, and $\ell_{\theta,i}$ is the score (gradient of the loss). The average $\bar{\psi} = n^{-1} \sum_i \hat{\psi}_i$ is the debiased estimator, and its standard error provides valid inference.

1.2 Running Example: H&M Fashion Demand

Our running application uses the H&M Personalized Fashion Recommendations dataset ([H&M Group, 2022](#))—a publicly available dataset of 1.4 million fashion transactions. We estimate a multinomial choice model for online dress purchases with heterogeneous price sensitivity across consumers.

The application demonstrates the full pipeline:

1. **Representation learning.** We train two-tower contrastive embeddings ([van den Oord et al., 2018](#)) to learn dense representations of consumers and products from purchase histories and visual features. Consumer embeddings capture latent taste dimensions; item embeddings capture visual and categorical product attributes.
2. **Choice model estimation.** Consumer embeddings become the covariates X that drive heterogeneity. Each consumer has their own price sensitivity $\theta_{\text{price}}(X_i)$ and taste parameters $\theta_{\text{style},k}(X_i)$. The neural network maps the 64-dimensional consumer embedding to these structural parameters.
3. **Influence function inference.** The `deep-inference` package handles cross-fitting, Lambda estimation, influence function assembly, and confidence interval computation. With a five-line custom loss function, the entire inference pipeline runs automatically.
4. **Counterfactual analysis.** We compute average price elasticities with valid confidence intervals and simulate the revenue impact of a 10% price increase—with uncertainty bands that properly account for the neural network estimation step.

1.3 What This Guide Covers

Section 2 develops the framework: the structural model, target functionals, why naive inference fails, the IF correction, and the three Lambda estimation regimes. Section 3 covers estimation in practice: data requirements, the algorithm step by step, cross-fitting, Lambda estimation, network architecture, diagnostics, and common pitfalls. Section 4 presents the full H&M application with results. Section 5 discusses extensions and other structural models. Appendix A provides annotated code and an installation guide.

Throughout, we emphasize the *practitioner’s* perspective: what choices must be made, what can go wrong, and what to look for in the output. We use the first person and aim for clarity over formality.

2 The Framework

This section lays out the FLM framework as a practitioner would encounter it: we describe the structural model, define target functionals, explain why naive neural network inference fails, present the influence function correction, and classify problems into three “regimes” that determine how a key quantity—the expected Hessian $\Lambda(x)$ —is handled.

2.1 Setup

We observe n i.i.d. observations (Y_i, T_i, X_i) where:

- Y_i is the outcome (chosen product, sales quantity, duration, etc.)
- T_i is the treatment or “action” variable (price, product attributes, dosage)
- $X_i \in \mathbb{R}^{d_x}$ are individual covariates that drive heterogeneity

A *structural model* specifies a loss function $\ell(y, t, \theta)$ parameterized by structural parameters $\theta \in \mathbb{R}^{d_\theta}$. The true parameters for individual i solve:

$$\theta^*(X_i) = \arg \min_{\theta} E[\ell(Y, T, \theta) \mid X = X_i] \quad (2)$$

Concrete example. In our H&M application:

- $Y_i \in \{0, 1, \dots, J-1\}$ is the chosen product from a set of J alternatives
- $T_i \in \mathbb{R}^{J \times K}$ contains the attributes of each alternative (log-price, style embeddings)
- $X_i \in \mathbb{R}^{64}$ is the consumer’s learned embedding
- $\theta^*(X_i) = (\beta_{\text{price}}(X_i), \beta_{\text{style},1}(X_i), \dots, \beta_{\text{style},5}(X_i))$ are the consumer’s taste parameters
- The loss is the multinomial logit negative log-likelihood:

$$\ell(y, t, \theta) = -V_y + \log \sum_{j=0}^{J-1} e^{V_j}, \quad V_j = x_j' \theta \quad (3)$$

The framework is *not* limited to discrete choice. Table 1 lists the structural models currently available in the **deep-inference** package, along with their loss functions and parameter dimensions.

Table 1: Available structural models in the **deep-inference** package.

Model	Loss $\ell(y, t, \theta)$	Link	d_θ
Linear	$(y - \alpha - \beta t)^2$	Identity	2
Logit	$\log(1 + e^\eta) - y\eta$	Logistic	2
Poisson	$e^\eta - y\eta$	Log	2
Gamma	$y/\mu + \log \mu$	Log	2
Multinomial	$-V_y + \log \sum_j e^{V_j}$	Softmax	$(J-1)+K$
Custom	Any differentiable $\ell(y, t, \theta)$	Any	User-specified

Note: $\eta = \alpha + \beta t$, $\mu = g^{-1}(\eta)$. The package includes 13 families total (Weibull, Gumbel, Tobit, NegBin, Probit, Beta, ZIP). Custom losses can be supplied via the `loss=` argument.

2.2 Target Functionals

We are usually not interested in $\theta^*(X_i)$ for every individual, but rather in some summary—an *average* of a function of θ^* across the population:

$$\mu^* = E[H(X, \theta^*(X), \tilde{t})] \quad (4)$$

where H is the *target function* and $\tilde{t}$ is an evaluation point. Common targets include:

Table 2: Target functionals for inference.

Target	$H(x, \theta, \tilde{t})$	Interpretation
Average parameter	θ_k	$E[\beta_{\text{price}}]$: mean price sensitivity
Average marginal effect	$g'(\alpha + \beta \tilde{t}) \cdot \beta$	Mean marginal effect at $\tilde{t}$
Elasticity	$\beta \cdot \tilde{t} \cdot (1 - p)$	Price elasticity of demand
Dose-response	$g(\alpha + \beta \tilde{t})$	Predicted outcome at $\tilde{t}$
Profit	$\tilde{t} \cdot g(\alpha + \beta \tilde{t})$	Expected revenue
Custom	Any $H(x, \theta, \tilde{t})$	User-specified

The Jacobian $H_\theta = \partial H / \partial \theta$ is required for the IF correction. For built-in targets, closed-form Jacobians are provided. For custom targets, the package computes them via automatic differentiation.

2.3 Why Naive Inference Fails

To estimate μ^* , we train a neural network $\hat{\theta}(X)$ by minimizing $\sum_i \ell(Y_i, T_i, \hat{\theta}(X_i))$ with sample splitting and then compute $\hat{\mu} = n^{-1} \sum_i H(X_i, \hat{\theta}(X_i), \tilde{t})$.

The *naive* standard error treats $\hat{\theta}$ as if it were the truth and computes $\text{SE}_{\text{naive}} = \text{sd}(H_i) / \sqrt{n}$. This is wrong because $\hat{\theta}$ is itself estimated with error, and neural network regularization introduces systematic bias.

The problem is quantitatively severe. In our simulations (Section 4 and Appendix A):

- Naive 95% CIs achieve only 0–20% actual coverage

- The naive SE is 3–10× too small
- The bias is toward zero (regularization shrinks $\hat{\theta}$)

This is not a small-sample problem—it persists at $n = 50,000$. The regularization bias is a *feature* of neural network training (it prevents overfitting) but becomes a *bug* for inference.

2.4 The Influence Function Correction

The FLM influence function correction removes the regularization bias. For each observation i , define:

$$\psi_i = H_i - H_{\theta,i} \cdot \Lambda_i^{-1} \cdot \ell_{\theta,i} \quad (5)$$

where:

- $H_i = H(X_i, \hat{\theta}(X_i), \hat{t})$ is the target at estimated parameters
- $H_{\theta,i} = \frac{\partial H}{\partial \theta} \Big|_{\hat{\theta}(X_i)} \in \mathbb{R}^{1 \times d_\theta}$ is the target Jacobian
- $\Lambda_i = E[\nabla_\theta^2 \ell(Y, T, \theta) \mid X = X_i] \Big|_{\hat{\theta}(X_i)} \in \mathbb{R}^{d_\theta \times d_\theta}$ is the expected Hessian
- $\ell_{\theta,i} = \nabla_\theta \ell(Y_i, T_i, \hat{\theta}(X_i)) \in \mathbb{R}^{d_\theta}$ is the score

The debiased estimator and its standard error are:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n \psi_i, \quad \widehat{\text{SE}} = \frac{\text{sd}(\psi_1, \dots, \psi_n)}{\sqrt{n}} \quad (6)$$

Intuition. The correction term $H_\theta \Lambda^{-1} \ell_\theta$ is an “adjustment for what the neural net got wrong.” The score ℓ_θ measures how far observation i ’s data is from the fitted model. The Hessian Λ converts this into parameter-space error. The Jacobian H_θ maps this to the target. When the neural net fits perfectly (scores near zero), the correction vanishes. When it makes systematic errors (regularization bias), the correction kicks in.

Connection to classical statistics. Equation (5) is a semiparametric influence function in the tradition of [Hampel \(1974\)](#) and [Newey and McFadden \(1994\)](#). The specific form arises from the “one-step correction” or “debiasing” approach used widely in semiparametric econometrics ([Chernozhukov et al., 2018](#)). The contribution of FLM is showing that this correction remains valid when the first-stage nuisance (the structural parameters) is estimated by a deep neural network.

2.5 Three Regimes for Lambda

The expected Hessian $\Lambda(X) = E[\nabla_\theta^2 \ell(Y, T, \theta) \mid X]$ depends on how the model and data are structured. Three cases arise:

Table 3: Three Lambda estimation regimes.

Regime	When	Lambda method	Cross-fitting
A	RCT, known F_T	Compute (MC integration)	2-way
B	Linear model	Analytic (closed-form)	2-way
C	Obs. + nonlinear	Estimate (ridge)	3-way

Regime A: Randomized experiment with known treatment distribution. If T is randomly assigned with known distribution F_T , and the Hessian does not depend on Y , then $\Lambda(X)$ can be computed via Monte Carlo integration: $\Lambda(X) = \int \nabla_{\theta}^2 \ell(y, t, \theta) dF_T(t)$. This is the simplest case. Examples: A/B tests, randomized pricing experiments.

Regime B: Linear model. For linear models, the Hessian is constant ($\nabla_{\theta}^2 \ell = 2$), so $\Lambda(X) = 2E[TT' | X]$, which can be estimated analytically. This avoids the need for three-way splitting.

Regime C: Observational data with nonlinear model. In most applied settings—including our H&M application—the Hessian depends on θ (e.g., through $p(1-p)$ in logit or softmax probabilities in multinomial logit). Since θ is estimated, we cannot simply compute Λ ; we must *estimate* it. FLM propose regressing the individual Hessians on X using a ridge regression. This requires a *three-way* sample split: one third for estimating $\hat{\theta}$, one third for estimating $\hat{\Lambda}$, and one third for assembling the IF. The package handles this automatically.

2.6 The Multinomial Choice Model

Our application uses a multinomial logit (conditional logit) model (McFadden, 1974; Train, 2009). Consumer i chooses from J alternatives, each described by K attributes $x_{ij} \in \mathbb{R}^K$. Utilities are:

$$V_{ij} = x'_{ij}\theta(X_i), \quad P(Y_i = j | X_i) = \frac{e^{V_{ij}}}{\sum_{m=1}^J e^{V_{im}}} \quad (7)$$

This is closely related to the BLP random coefficients model (Berry et al., 1995; Nevo, 2000). The key differences are:

1. BLP estimates a distribution (e.g., $\theta \sim N(\bar{\theta}, \Sigma)$) using aggregate market-level data. FLM estimates $\theta(X_i)$ for each individual using individual-level data.
2. BLP uses GMM with instruments. FLM uses neural networks with influence function correction.
3. BLP requires solving a fixed-point contraction mapping for each market. FLM requires only a forward pass through the neural network.

The connection between two-tower embeddings and the FLM framework is natural: the consumer embedding X_i captures the latent heterogeneity dimensions, and the neural network maps $X_i \rightarrow \theta(X_i)$. The item attributes T_i include prices and (PCA-reduced) item embeddings.

2.7 Available Models and Targets

The `deep-inference` package provides 13 built-in model families (Table 1) and 10+ target functionals (Table 2). For models not in the built-in library, users can supply a custom loss function:

```

1  import torch
2  from deep_inference import inference
3
4  def my_loss(y, t, theta):
5      """Custom structural loss."""
6      mu = torch.exp(theta[0] + theta[1] * t) # log-link
7      return mu - y * torch.log(mu)          # Poisson NLL
8
9  result = inference(Y, T, X, loss=my_loss, theta_dim=2,
10                    target_fn=lambda x, th, tt: th[1])

```

The package automatically determines whether the Hessian depends on θ (triggering Regime C and three-way splitting) and computes all required derivatives via automatic differentiation. The user needs only the loss function and the target.

3 Estimation in Practice

This section is the heart of the guide. We walk through every step of the estimation procedure, discuss the choices practitioners must make, and flag the pitfalls that can silently invalidate results.

3.1 Data Requirements

The method requires individual-level data in triplets (Y_i, T_i, X_i) :

- **Outcomes Y :** The variable being modeled. Can be continuous (sales, duration), binary (purchase, click), count (visits), or discrete choice (chosen alternative index).
- **Treatments/actions T :** The variable(s) whose effect varies across individuals. For discrete choice, T contains the attributes of all alternatives (prices, features). For treatment effects, T is the treatment indicator or dose.
- **Covariates X :** The individual characteristics that drive heterogeneity. These are the neural network’s inputs. Higher-dimensional X allows the network to capture richer heterogeneity patterns, but requires more data.

Sample size considerations. Neural networks are data-hungry. As a rough guide:

- $n \geq 2,000$ for binary logit with $d_\theta = 2$
- $n \geq 5,000$ for multinomial logit with $d_\theta \leq 6$
- $n \geq 8,000$ for multinomial logit with $d_\theta = 4$ and three-way split

The three-way split (Regime C) reduces effective training data to about 60% of n , so more data is needed than for Regime A or B models.

Choice set construction. For multinomial choice models, each observation requires a full choice set: the chosen item plus $J - 1$ alternatives. When the full choice set is too large (e.g., 100,000 products), sample $J - 1$ random alternatives per occasion. This is consistent with McFadden’s sampling theorem and standard in the IO literature (Train, 2009). We use $J = 20$ in our application.

3.2 The Algorithm

The estimation proceeds in five steps, all handled internally by the `inference()` function:

- Step 0. Prepare data.** Encode T as a numeric array with consistent shape across observations. For multinomial logit: pack alternative attributes as $(n, J \times K)$.
- Step 1. Cross-fitting.** Split data into K folds. For each fold, train the structural network $\hat{\theta}(X)$ on the remaining $K - 1$ folds and predict on the held-out fold. This produces out-of-sample estimates $\hat{\theta}(X_i)$ for all i .
- Step 2. Estimate Lambda.** Compute individual Hessians $\nabla_\theta^2 \ell(Y_i, T_i, \hat{\theta}(X_i))$ via autodiff. Regress these Hessians on X using ridge regression to obtain $\hat{\Lambda}(X_i) = E[\nabla_\theta^2 \ell \mid X_i]$.
- Step 3. Compute scores and Jacobians.** Evaluate the score $\ell_\theta(Y_i, T_i, \hat{\theta}(X_i))$ and target Jacobian $H_\theta(X_i, \hat{\theta}(X_i), \tilde{t})$ at each observation.
- Step 4. Assemble ψ and compute SE.** Combine the pieces: $\psi_i = H_i - H_{\theta,i} \hat{\Lambda}_i^{-1} \ell_{\theta,i}$. Compute $\hat{\mu} = \bar{\psi}$ and $\widehat{\text{SE}} = \text{sd}(\psi)/\sqrt{n}$.

In code, this entire pipeline is a single function call:

```

1  from deep_inference import inference
2
3  result = inference(Y, T, X, loss=my_loss, theta_dim=6,
4                    target_fn=my_target,
5                    hessian_depends_on_theta=True,
6                    n_folds=50, epochs=300, patience=50)
7
8  print(f"Estimate: {result.mu_hat:.4f} +/- {result.se:.4f}")
9  print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")

```

3.3 Cross-Fitting Details

Cross-fitting is essential for valid inference. It prevents the neural network from “seeing” the same data used to evaluate the influence function, eliminating a bias term that would otherwise require undersmoothing conditions.

Number of folds. We recommend $K = 50$ folds. With fewer folds, each training set is smaller, potentially degrading $\hat{\theta}$. With more folds, computational cost increases linearly. In our experience, $K = 50$ provides a good balance for $n \geq 2,000$. At smaller sample sizes, $K = 20$ may be necessary.

Two-way vs. three-way splitting. When the Hessian depends on θ (Regime C), estimating Λ on the same data used to estimate θ would introduce dependence. The three-way split resolves this: within each fold, the data is further split so that $\hat{\theta}$ and $\hat{\Lambda}$ are estimated on independent subsets. The effective training sample for $\hat{\theta}$ drops to about 60% of n .

Practical implication. Three-way splitting is triggered automatically when the package detects that the Hessian depends on θ . For the multinomial logit, this is always the case (softmax probabilities depend on θ). The practitioner does not need to manage the splitting manually, but should be aware that more data is needed than for linear models.

3.4 Lambda Estimation

Lambda estimation is the most delicate step in the pipeline. An incorrect $\hat{\Lambda}$ directly corrupts the standard errors.

Table 4: Lambda estimation methods: comparison on logit simulation ($n = 5,000$, $M = 50$).

Method	Corr. w/ Oracle	Coverage	SE Ratio	Notes
Ridge ($\alpha = 1000$)	0.508	96%	0.91	Default, recommended
Aggregate	0.000	95%	1.00	Ignores X -dependence
LGBM	0.978	96%	1.00	Good accuracy
MLP	0.997	67%	0.65	Never use

Ridge (default). Ridge regression with strong regularization ($\alpha = 1000$) pulls $\hat{\Lambda}(X)$ toward the population mean Hessian. This sacrifices some accuracy in estimating heterogeneity in Λ but produces reliable standard errors. The high regularization parameter may seem counterintuitive—but recall that Lambda is not the object of interest; it is a nuisance quantity in the IF formula. We only need it to be “close enough” for valid inference.

MLP: a cautionary tale. An MLP Lambda estimator achieves the highest correlation with the oracle (0.997) yet produces catastrophically invalid standard errors (67% coverage). The problem

is overfitting: the MLP captures noise in the individual Hessians, producing $\hat{\Lambda}(X)$ estimates that are too extreme. Since Λ^{-1} appears in the IF formula, extreme Lambda values create extreme ψ_i values, inflating the variance estimator. The lesson: *high correlation with the oracle does not guarantee valid inference.*

3.5 Network Architecture

The structural network maps $X \rightarrow \theta(X)$ via a multi-layer perceptron. Key settings:

- **Hidden dimensions:** [64, 32] default. Two layers with decreasing width. Rarely needs tuning. Larger networks ([128, 64, 32]) may help with very rich heterogeneity patterns but require more data.
- **Learning rate:** 0.01. Standard for Adam optimizer with structural losses.
- **Epochs:** 200 for binary models, 300 for multinomial. The network needs enough training to learn the structural relationship.
- **Patience:** **At least 50 for multinomial models.** This is the number of epochs to wait without improvement before early stopping. With three-way splitting, the validation set is small and noisy, causing premature stopping if patience is too low. With `patience=10` (a common default), the network stops at epoch 15–20 and severely underfits. This is the single most common pitfall we have encountered.
- **Dropout:** 0.1 (fixed). Mild regularization to prevent overfitting.

3.6 Diagnostics

After running `inference()`, several diagnostic quantities should be checked:

1. **Minimum Lambda eigenvalue** $> 10^{-4}$. If the smallest eigenvalue of $\hat{\Lambda}$ is too close to zero, the Hessian is nearly singular. This typically means the model is poorly identified or the sample size is too small. The package adds a ridge regularization (10^{-4}) to prevent numerical issues, but a warning is printed.
2. **Correction ratio.** The ratio $\text{Var}(\text{correction})/\text{Var}(H_i)$ measures how much the IF correction contributes. For binary logit, values of 2–3 are normal. For multinomial logit, values of 70–90 are normal due to the higher-dimensional θ . Do not be alarmed by large correction ratios for multinomial models.
3. **SE ratio** $\in [0.7, 1.5]$. In simulation studies where the truth is known, the ratio of estimated SE to empirical SE should be near 1. Values below 0.7 suggest underestimated uncertainty; values above 1.5 suggest overestimation. On real data, this diagnostic is not directly available, but the other checks provide confidence.

4. **z -score distribution.** If running multiple targets or a simulation study, the z -scores $(\hat{\mu} - \mu^*)/\text{SE}$ should have mean ≈ 0 and standard deviation ≈ 1 . Departures indicate systematic bias or SE miscalibration.

3.7 Common Pitfalls

1. **Patience too low.** This is the number one cause of bad results. With three-way splitting, the validation set is small (about 7% of data with $K = 50$), making the validation loss noisy. Low patience triggers early stopping before the network has learned the structural relationship. **Fix:** Set `patience=50` or higher for multinomial models.
2. **MLP Lambda.** Using an MLP to estimate $\Lambda(X)$ produces beautiful correlation with the oracle but invalid standard errors. **Fix:** Use `ridge` (default) or `lgbm`.
3. **Sample size too small.** Three-way splitting with $K = 50$ folds means each fold has $n/50$ observations, of which 60% are used for training. At $n = 2,000$, that is 24 observations per fold for training—borderline for a neural network with $d_\theta = 6$. **Fix:** Use $n \geq 5,000$ for multinomial models.
4. **Ignoring diagnostics.** The package prints warnings for suspicious Lambda eigenvalues and correction ratios. Do not suppress these warnings. They are the first sign that something is wrong.
5. **Non-differentiable loss.** The IF correction requires computing the Hessian $\nabla_\theta^2 \ell$ via automatic differentiation. If the loss function contains non-differentiable operations (`int()`, `if/else` on tensors), the autodiff will fail. **Fix:** Use `torch` operations throughout the loss function.
6. **Data-dependent indexing.** For multinomial logit with reordered choice sets (chosen alternative always at index 0), use `V[0]` instead of `V[int(y)]`. The `int()` call breaks `torch.vmap`, which is used internally for batched Hessian computation.

3.8 Quick Start: Binary Logit

For readers who want to try the method before diving into the multinomial application, here is a minimal binary logit example:

```

1 import numpy as np
2 from deep_inference import inference
3
4 # Simulated data
5 np.random.seed(42)
6 n = 5000
7 X = np.random.randn(n, 5)           # Covariates
8 T = np.random.randn(n)              # Treatment

```

```

9  beta_true = 0.5 + 0.3 * X[:, 0]    # Heterogeneous effect
10 p = 1 / (1 + np.exp(-(1.0 + beta_true * T)))
11 Y = np.random.binomial(1, p).astype(float)
12
13 # Run inference
14 result = inference(Y, T, X, model='logit', target='beta')
15 print(f"E[beta] = {result.mu_hat:.3f} +/- {result.se:.3f}")
16 print(f"95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")
17 # Expected: E[beta] ~ 0.5 with valid coverage

```

The entire IF pipeline runs in about 2 minutes on a laptop CPU for $n = 5,000$.

4 Application: H&M Fashion Demand

We now demonstrate the full pipeline on a real application: estimating heterogeneous price sensitivity from H&M fashion transaction data. The application illustrates every step discussed in the previous sections and produces economic objects—elasticities, counterfactual revenues—with valid confidence intervals.

4.1 Data

The H&M Personalized Fashion Recommendations dataset ([H&M Group, 2022](#)) contains 1.4 million transactions from 369,000 customers purchasing 105,000 items between 2018 and 2020. We focus on online dress purchases in 2019–2020, yielding approximately 5,000 consumers with at least 5 purchases and 90 available items.

Each transaction records the consumer ID, item ID, purchase date, and price. Item metadata includes product type, color, department, and garment group. Crucially for our application, the dataset also provides product images, enabling visual feature extraction.

4.2 Learning Representations

The first step is representation learning. We need dense embeddings for consumers and products that capture relevant variation in preferences and attributes.

Two-tower architecture. We use a two-tower contrastive learning approach ([van den Oord et al., 2018](#); [Yi et al., 2019](#)). The *user tower* maps consumer features (ID embedding, purchase history, demographics) to a 64-dimensional embedding $u_i \in \mathbb{R}^{64}$. The *item tower* maps product features (CLIP visual embedding ([Radford et al., 2021](#)), categorical features) to a 64-dimensional embedding $v_j \in \mathbb{R}^{64}$.

InfoNCE training. The towers are trained jointly using the InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(u'_i v_{j^+} / \tau)}{\sum_{j \in \mathcal{B}} \exp(u'_i v_j / \tau)} \quad (8)$$

where j^+ is the purchased item, $\mathcal{B}$ is a mini-batch of items (serving as negatives), and $\tau = 0.1$ is the temperature. Training runs for 50 epochs with batch size 1024.

Output. The trained towers produce:

- `user_embeddings.npy`: $(5,000 \times 64)$ consumer embeddings
- `item_embeddings.npy`: (90×64) item embeddings

These embeddings are trained on a separate data period (T1) from the choice model estimation period (T2), so they can be treated as fixed covariates—a key assumption for the IF correction to be valid.

4.3 Mapping to the FLM Framework

The mapping from the two-tower output to the FLM framework is:

- X_i : Consumer embedding (64D). This determines heterogeneity in structural parameters.
- T_i : Packed alternative attributes. For each choice occasion, we sample $J = 20$ alternatives (1 chosen + 19 random). Each alternative has $K = 6$ attributes: log-price and 5 PCA dimensions of the item embedding ($64D \rightarrow 5D$ via PCA, capturing $\sim 70\%$ of variance). The packed array is $T_i \in \mathbb{R}^{120}$ (reshaped to 20×6 inside the loss).
- $Y_i = 0$: The chosen alternative is always reordered to position 0.
- $\theta(X_i) = (\beta_{\text{price}}, \beta_{\text{pca}_1}, \dots, \beta_{\text{pca}_5}) \in \mathbb{R}^6$: Consumer i 's taste parameters.

Why PCA? The raw item embedding is 64-dimensional, which would require $d_\theta = 65$ (including price). This is too many parameters for stable Lambda estimation with the sample sizes available. PCA reduces to 5 dimensions while retaining the major axes of product variation. The resulting $d_\theta = 6$ is small enough for stable estimation.

No alternative-specific constants. Unlike the standard multinomial logit with alternative-specific intercepts (α_j) , we use only attribute-based utilities: $V_{ij} = x'_{ij} \theta(X_i)$. With sampled choice sets, alternative identities change across occasions, making ASCs ill-defined. The price and embedding features provide sufficient variation for identification.

Custom loss function. The loss is defined in five lines:

```

1  import torch
2
3  def mnl_loss(y, t, theta):
4      J, K = 20, 6
5      x = t.reshape(J, K)          # Unpack attributes
6      V = x @ theta                # Utilities
7      return -V[0] + torch.logsumexp(V, dim=0) # -log P(chosen)

```

Calling inference. The complete inference call:

```

1  from deep_inference import inference
2
3  def price_target(x, theta, t_tilde):
4      return theta[0] # beta_price
5
6  result = inference(
7      Y=Y, T=T, X=X,
8      loss=mnl_loss, theta_dim=6,
9      hessian_depends_on_theta=True,
10     target_fn=price_target,
11     n_folds=50, epochs=300, patience=50,
12 )
13
14 print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")

```

4.4 Results

4.4.1 Simulation Validation

Before applying the method to real data, we validate it on a simulation DGP that mirrors the application specification ($J = 20$, $K = 6$, $d_x = 10$). We run $M = 50$ Monte Carlo replications at $n = 5,000$.

Table 5: Simulation validation: IF coverage for custom MNL loss.

Metric	Value	Target
Coverage (95% CI)	[to be filled]	90–99%
SE ratio	[to be filled]	0.7–1.5
Bias	[to be filled]	< 0.05
z -mean	[to be filled]	(−0.3, 0.3)
z -std	[to be filled]	0.7–1.5

Note: Results from application/06_simulation_study.py with $M = 50$, $n = 5,000$, $J = 20$, $K = 6$.

The simulation confirms that the custom MNL loss with the `inference()` API achieves valid coverage, providing confidence before proceeding to real data.

4.4.2 Main Inference Results

Table 6 reports the estimated average structural parameters with IF-corrected standard errors. The key result is $E[\beta_{\text{price}}]$: the average price sensitivity across consumers.

Table 6: Estimated average structural parameters with IF-corrected standard errors.

Parameter	$\hat{\mu}$	SE (IF)	SE (Naive)	Ratio	95% CI
$E[\beta_{\text{price}}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]
$E[\beta_{\text{pca}_1}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]
$E[\beta_{\text{pca}_2}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]
$E[\beta_{\text{pca}_3}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]
$E[\beta_{\text{pca}_4}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]
$E[\beta_{\text{pca}_5}]$	[to fill]	[to fill]	[to fill]	[to fill]	[to fill]

Note: IF SE accounts for neural network estimation uncertainty. Ratio = IF SE / Naive SE. Results from `application/03_inference.py` with `n_folds = 50`, `epochs = 300`, `patience = 50`.

The IF standard errors are systematically larger than naive SEs, reflecting the additional uncertainty from neural network estimation. This is exactly the pattern predicted by theory: naive SEs ignore the estimation step and produce overconfident intervals.

4.4.3 Heterogeneity in Price Sensitivity

The neural network estimates individual-level $\hat{\beta}_{\text{price}}(X_i)$ for each consumer. The distribution of these estimates reveals substantial heterogeneity in price sensitivity across the consumer population.

4.5 Counterfactuals

With valid confidence intervals on the structural parameters, we can construct counterfactuals with properly quantified uncertainty.

Own-price elasticities. For each consumer, the own-price elasticity of the chosen item is:

$$\eta_i = \hat{\beta}_{\text{price}}(X_i) \cdot (1 - \hat{P}_{i,\text{chosen}}) \quad (9)$$

where $\hat{P}_{i,\text{chosen}}$ is the predicted choice probability. The IF provides pointwise uncertainty on $\hat{\beta}_{\text{price}}$, which propagates to elasticity uncertainty via the delta method.

Revenue impact of a 10% price increase. We simulate a uniform 10% price increase for the chosen item and compute the change in expected revenue:

$$\Delta R = E \left[\frac{p \cdot (1 + \delta) \cdot \hat{P}_{\text{new}}}{p \cdot \hat{P}_{\text{base}}} - 1 \right] \quad (10)$$

where $\delta = 0.10$. The IF confidence intervals on $\hat{\beta}_{\text{price}}$ translate directly into uncertainty bands on the revenue impact.

Personalized pricing. Because $\hat{\beta}_{\text{price}}(X_i)$ varies across consumers, the revenue-maximizing price differs by consumer segment. Consumers in the least price-sensitive quintile can absorb larger price increases, while the most sensitive quintile would see significant demand drops. The IF correction ensures that these segment-level estimates come with valid confidence intervals, preventing overconfident pricing recommendations.

5 Concluding Remarks

We have presented a step-by-step guide to the FLM framework for deep learning with individual heterogeneity. Using an H&M fashion demand application, we demonstrated the full pipeline: from representation learning through contrastive embeddings, to choice model estimation with a custom neural network loss, to influence-function-corrected inference on economic objects like average price sensitivity and counterfactual revenue impacts.

Three lessons stand out for practitioners:

1. **Influence function correction is not optional.** Naive neural network standard errors are systematically too small—often by a factor of 3–10 $\times$. The resulting confidence intervals can have 0–20% actual coverage when the nominal level is 95%. The IF correction is the difference between “suggestive evidence” and valid statistical inference.
2. **Lambda estimation requires restraint.** The temptation to use a powerful estimator (MLP) for the expected Hessian $\Lambda(X)$ must be resisted. Heavy regularization (ridge with $\alpha = 1000$) produces better inference than a flexible estimator that overfits the Hessians. Coverage, not correlation with the oracle, is the right metric.
3. **Patience matters.** For nonlinear models with three-way cross-fitting, the early stopping patience parameter is critical. Too-low patience causes premature stopping and underfitting. We recommend `patience=50` as a minimum for multinomial models.

5.1 Other Structural Models

The framework extends directly to other structural models. The `deep-inference` package includes:

- **Poisson:** Count data (visits, clicks). Log-link: $\lambda = \exp(\alpha + \beta t)$.

- **Weibull/Gumbel:** Duration and survival data. Heterogeneous hazard rates.
- **Gamma:** Positive continuous data (expenditures). Log-link.
- **Tobit:** Censored data. Threshold effects with heterogeneous sensitivity.
- **Negative binomial:** Overdispersed counts. Allows for unobserved heterogeneity in rates.
- **Zero-inflated Poisson:** Count data with excess zeros (e.g., purchase frequency).
- **Combinatorial:** Multi-treatment experiments with multiple interaction effects (Ye et al., 2025).

For any model not in the built-in library, the custom loss interface (`loss=`) allows practitioners to define their own structural model and get valid inference with no additional implementation effort.

5.2 Other Targets

Beyond average parameters, the package supports:

- **Average marginal effects:** $E[g'(\theta't) \cdot \beta]$ at an evaluation point.
- **Dose-response:** $E[g(\theta'\tilde{t})]$ — average predicted outcome at a counterfactual treatment level (Colangelo and Lee, 2026).
- **Profit/Revenue:** $E[\tilde{t} \cdot g(\theta'\tilde{t})]$ — expected revenue at a counterfactual price (Dubé and Misra, 2023).
- **Tail probability:** $E[P(Y > c \mid \theta, \tilde{t})]$ — exceedance probability.
- **Conditional variance:** $E[\text{Var}(Y \mid \theta, \tilde{t})]$ — model uncertainty.
- **Custom targets:** Any differentiable function $H(x, \theta, \tilde{t})$ via the `target_fn=` argument.

5.3 Custom Models via `model_from_loss()`

For researchers working with non-standard structural models, the custom loss interface is the most important feature. The key requirements are:

1. The loss function must be written in PyTorch and be twice-differentiable.
2. Avoid `int()`, `if/else` on tensor values, or other non-differentiable operations.
3. Specify `theta_dim` explicitly.
4. If the Hessian depends on θ (nonlinear models), set `hessian_depends_on_theta=True` to ensure three-way splitting.

5.4 Computational Considerations

- **Runtime:** For $n = 50,000$ with $K = 50$ folds: approximately 10–30 minutes on CPU, 2–5 minutes on GPU.
- **Memory:** The main bottleneck is storing $n \times d_\theta \times d_\theta$ Hessians for Lambda estimation. At $n = 50,000$ and $d_\theta = 6$, this is about 7 MB—negligible.
- **Parallelism:** Cross-fitting folds are independent and can be parallelized. The current implementation is sequential but GPU-accelerated within each fold.

5.5 Three Regimes Revisited

We close by noting that the choice of regime (A, B, or C) has practical implications beyond Lambda estimation:

- **Regime A** (RCTs with known F_T): The simplest case. Lambda can be computed analytically. Two-way splitting suffices. If you have a randomized experiment, use it.
- **Regime B** (linear models): Lambda has a closed form. Two-way splitting suffices. Fast and reliable.
- **Regime C** (the general case): Most applied settings fall here. Three-way splitting is required, patience must be high, and sample size requirements are larger. But the method works—our simulation confirms valid coverage for the multinomial logit with $n = 5,000$ and $d_\theta = 6$.

The `deep-inference` package detects the regime automatically and handles all the implementation details. The practitioner’s job is to specify the model, prepare the data, and check the diagnostics.

References

- Berry, Steven, James Levinsohn, and Ariel Pakes, “Automobile prices in market equilibrium,” *Econometrica*, 1995, *63* (4), 841–890.
- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins, “Double/debiased machine learning for treatment and structural parameters,” *The Econometrics Journal*, 2018, *21* (1), C1–C68.
- Colangelo, Kyle and Ying-Ying Lee, “Double debiased machine learning for continuous treatments,” *Journal of Business & Economic Statistics*, 2026. Forthcoming.
- Dubé, Jean-Pierre and Sanjog Misra, “Personalized pricing and consumer welfare,” *Journal of Political Economy*, 2023, *131* (1), 131–189.

- Farrell, Max H, Tengyuan Liang, and Sanjog Misra**, “Deep neural networks for estimation and inference,” *Econometrica*, 2021, *89* (1), 181–213.
- , —, and —, “Deep learning for individual heterogeneity: An automatic inference framework,” *Econometrica*, 2025. Forthcoming.
- Hampel, Frank R**, “The influence curve and its role in robust estimation,” *Journal of the American Statistical Association*, 1974, *69* (346), 383–393.
- H&M Group**, “H&M Personalized Fashion Recommendations,” Kaggle Competition 2022.
- McFadden, Daniel**, “Conditional logit analysis of qualitative choice behavior,” *Frontiers in Econometrics*, 1974, pp. 105–142.
- Nevo, Aviv**, “A practitioner’s guide to estimation of random-coefficients logit models of demand,” *Journal of Economics & Management Strategy*, 2000, *9* (4), 513–548.
- Newey, Whitney K and Daniel McFadden**, “Large sample estimation and hypothesis testing,” *Handbook of Econometrics*, 1994, *4*, 2111–2245.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandeep Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark et al.**, “Learning transferable visual models from natural language supervision,” in “International Conference on Machine Learning” 2021, pp. 8748–8763.
- Train, Kenneth E**, *Discrete Choice Methods with Simulation*, 2nd ed., Cambridge University Press, 2009.
- van den Oord, Aaron, Yazhe Li, and Oriol Vinyals**, “Representation learning with contrastive predictive coding,” in “Advances in Neural Information Processing Systems” 2018.
- Ye, Shuangning et al.**, “Deep learning for combinatorial causal inference,” *Management Science*, 2025. Forthcoming.
- Yi, Xinyang, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi**, “Sampling-bias-corrected neural modeling for large corpus item recommendations,” in “Proceedings of the 13th ACM Conference on Recommender Systems” 2019, pp. 269–277.

A Implementation Details

This appendix provides annotated code for the H&M application and a diagnostics interpretation guide.

A.1 Installation

```
pip install deep-inference
```

The package requires Python ≥ 3.8 , PyTorch ≥ 1.12 , NumPy, and scikit-learn (for ridge Lambda estimation). GPU support is automatic via PyTorch’s CUDA backend.

A.2 Data Preparation

The data preparation script loads pre-trained embeddings, applies PCA to item embeddings, and constructs choice sets.

```
1  import numpy as np
2  from sklearn.decomposition import PCA
3
4  # Load pre-trained two-tower embeddings
5  user_emb = np.load("artifacts/user_embeddings.npy") # (5000, 64)
6  item_emb = np.load("artifacts/item_embeddings.npy") # (90, 64)
7  prices = np.load("artifacts/ref_prices.npy") # (90,)
8
9  # PCA on item embeddings: 64D -> 5D
10 pca = PCA(n_components=5)
11 item_pca = pca.fit_transform(item_emb)
12 print(f"Variance explained: {pca.explained_variance_ratio_.sum():.1%}")
13
14 # Build item attribute matrix: [log_price, pca_1, ..., pca_5]
15 log_prices = np.log(prices + 1e-6)
16 item_attrs = np.column_stack([log_prices, item_pca]) # (90, 6)
17
18 # For each purchase occasion, construct choice set
19 J = 20 # 1 chosen + 19 random alternatives
20 K = 6 # log_price + 5 PCA dims
21 rng = np.random.RandomState(42)
22
23 Y_list, T_list, X_list = [], [], []
24
25 for user_id, chosen_item_id in transactions:
26     # Consumer embedding
27     X_list.append(user_emb[user_id])
28
```

```

29     # Pack chosen item first, then J-1 random alternatives
30     t_row = np.zeros(J * K)
31     t_row[:K] = item_attrs[chosen_item_id]
32
33     others = np.setdiff1d(np.arange(len(item_attrs)), [chosen_item_id])
34     sampled = rng.choice(others, J - 1, replace=False)
35     for j, alt_id in enumerate(sampled):
36         t_row[(j+1)*K : (j+2)*K] = item_attrs[alt_id]
37
38     T_list.append(t_row)
39     Y_list.append(0.0) # Chosen is always index 0
40
41 Y = np.array(Y_list, dtype=np.float32)
42 T = np.array(T_list, dtype=np.float32)
43 X = np.array(X_list, dtype=np.float32)
44 np.save("data/Y.npy", Y)
45 np.save("data/T.npy", T)
46 np.save("data/X.npy", X)

```

A.3 Inference Call

The inference call is concise once the data is prepared:

```

1  import torch
2  import numpy as np
3  from deep_inference import inference
4
5  # Load prepared data
6  Y = np.load("data/Y.npy")      # (n,)
7  T = np.load("data/T.npy")      # (n, 120)
8  X = np.load("data/X.npy")      # (n, 64)
9
10 # Custom MNL loss (5 lines)
11 J, K = 20, 6
12
13 def mnl_loss(y, t, theta):
14     x = t.reshape(J, K)          # (20, 6) attribute matrix
15     V = x @ theta                 # (20,) utilities
16     return -V[0] + torch.logsumexp(V, dim=0)
17
18 # Target: average price sensitivity
19 def price_target(x, theta, t_tilde):
20     return theta[0]
21
22 # Run inference

```

```

23 result = inference(
24     Y=Y, T=T, X=X,
25     loss=mn1_loss,
26     theta_dim=K,                # 6 parameters per consumer
27     hessian_depends_on_theta=True, # Softmax probs depend on theta
28     hessian_depends_on_y=False,    # Fisher information Hessian
29     target_fn=price_target,
30     n_folds=50,                  # Cross-fitting folds
31     epochs=300,                  # Training epochs
32     patience=50,                 # CRITICAL for 3-way split
33     hidden_dims=[64, 32],        # Network architecture
34     lr=0.01,                     # Learning rate
35     verbose=True,                # Print progress
36 )
37
38 # Results
39 print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")
40 print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")
41
42 # Diagnostics
43 print(f"Regime: {result.diagnostics.get('regime', 'C')}")
44 print(f"Lambda method: {result.diagnostics.get('lambda_method')}")

```

A.4 Running All Parameters

To estimate all K parameters, loop over target indices:

```

1 param_names = ["log_price", "pca_1", "pca_2", "pca_3", "pca_4", "pca_5"]
2
3 for k, name in enumerate(param_names):
4     target_fn = lambda x, theta, t_tilde, idx=k: theta[idx]
5
6     result = inference(
7         Y=Y, T=T, X=X,
8         loss=mn1_loss, theta_dim=K,
9         hessian_depends_on_theta=True,
10        target_fn=target_fn,
11        n_folds=50, epochs=300, patience=50,
12    )
13    print(f"E[{name}] = {result.mu_hat:.4f} +/- {result.se:.4f}")

```

A.5 Diagnostics Interpretation Guide

1. **Check the regime.** The output should say “Regime C” for our multinomial model. If it says Regime A or B, something is wrong with the Hessian detection.
2. **Check Lambda eigenvalues.** If a warning about small eigenvalues appears, the model may be poorly identified. Consider: (a) reducing d_θ (more PCA compression), (b) increasing n , or (c) using a different Lambda method.
3. **Check the correction ratio.** For multinomial logit, values of 70–90 are normal. For binary logit, values of 2–3 are normal. Very large values (> 200) suggest Lambda estimation problems.
4. **Verify with simulation.** Before trusting results on real data, run the simulation study (`application/06.simulation_study.py`) with a DGP matching your application specification. If simulation coverage is near 95%, you can be confident in the real-data results.
5. **Compare IF to naive SE.** The IF SE should be larger than the naive SE. If it is smaller, something is wrong. The ratio depends on the model: $2\text{--}5\times$ for binary logit, potentially larger for multinomial.

A.6 Companion Repository

The complete code for the application is available at:

<https://github.com/rawatpranjal/deep-inference/tree/main/application>

The repository includes:

- Pre-trained embedding artifacts (via symlink)
- Data preparation, inference, comparison, and counterfactual scripts
- Simulation study with coverage validation
- Figure generation for the paper
- The `deep-inference` package source code